

claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-a409af805bd18f445.jsonl`  
- SHA-256: `993469f5d41845b93c014339ed0aa325dfbc097a92ec7e42ec16026dbef5e07a`  
- Source modified: `2026-07-02T02:59:38+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_41c8dc8a-8a4`  
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`
```

Transcript

1. user (2026-07-02T02:58:05.135Z)

You are an adversarial verifier in a tech-debt audit. Your default stance: REFUTE. Today is 2026-07-02.

A finder claims the following about the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer:

```
{  
  "title": "/api/process re-runs the paid Gemini segmenter for an already-processed video (re-bill on refresh/'Try again') ? still unfixed at HEAD",  
  "category": "critical-fix",  
  "severity": "P1",  
  "file": "backend/main.py",  
  "line": 1172,  
  "evidence": "backend/main.py:1171-1172 computes video_id and `was_reingest` = db.get_video(video_id) is not None`, but was_reingest is only used for the report/destroy-after-confirm; there is NO reuse/skip branch ? the segmenter always runs (main.py:1251-1255). Verified against gh: issue #340 OPEN ('re-runs the full (paid) segmenter anyway'), plus #330 (MD5 segmentation reuse) OPEN.",  
  "why_it_matters": "A page refresh or SPA 'Try again' on an already-indexed video (same MD5) re-bills Gemini (~$0.10/8-min match) and burns a multi-minute compute run for zero new information. Live-triggered once in the alpha (2026-06-22, per #340).",  
  "proposed_fix": "In _execute_processing, when db.get_video(video_id) exists with status='processed' and existing play_segments (and the request doesn't explicitly force reprocess), return the stored segments as the job result instead of dispatching the segmenter; add a `force` flag to ProcessRequest for deliberate re-ingest.",  
  "effort": "M",  
  "tracked_in": "Khelsutra/badminton-highlight-indexer#340 (also #330)"  
}
```

Repo root candidates: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer, C:/Users/avidu/Projects/khelsutra-guru/khelsutra, C:/Users/avidu/Projects/khelsutra-guru/sports-data-collector, C:/Users/avidu/Projects/khelsutra-guru/rally-annotator, C:/Users/avidu/Projects/khelsutra-guru/sports-obsreport (pick from the finding's repo field; non-git dirs: 'C:/Users/avidu/Projects/khelsutra-guru/Annotation Setup', C:/Users/avidu/Projects/khelsutra-guru/khelsutra-screencast).

VERIFY against the CURRENT working tree (just pulled to origin master/main): (1) does the cited code actually exist and behave as claimed at file:line? (2) was it already fixed by a recent commit (git log --oneline -15 -- <file>)? (3) is the severity honest - construct the concrete failure scenario or downgrade; (4) is the proposed fix sane and correctly scoped; (5) is it owner-gated per: alpha/serving go-live, Studio P10/P11 corpus-promotion/training, product/strategy/licensing, real cloud spend, counsel ToS/DPA?

HARD RULES: READ-ONLY (no file modifications, no installs, no test-suite runs). gh read-only commands allowed. Your reasoning MUST cite file:line you actually read. CONFIRMED requires you personally saw the defect/debt in current code; if you cannot reproduce the claim from the code, say REFUTED; if a commit fixed it, ALREADY_FIXED.

2. user (2026-07-02T02:58:14.572Z)

Already up to date.

master

e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
8b16cf4 docs: reflect today's storage endpoints + the corpus/user-video boundary (#411)

3. user (2026-07-02T02:58:20.258Z)

```
1130
1131
1132 def _execute_processing(
1133     config=None, db=None, req=None, progress=None
1134 ) -> Dict[str, Any]:
1135     """The full hash ? validate ? segment ? report pipeline for one video.
1136
1137     Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
1138     module-level ``global_config``/``db_instance`` (backward-compatible with existing
1139     tests that pass only ``req``). Route handlers always pass explicit params; older
1140     test code that calls ``_execute_processing(req)`` still works.
1141
1142     ``progress`` is an optional callable(stage: str) used to surface coarse job
1143 progress.
1144     Raises on unexpected internal errors ? the caller records those as a job failure
1145     """
1146     # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
1147     # shift positional args ? req arrives in position 0 (config), progress in position 1
1148     (db),
1149     if config is not None and not isinstance(config, AppConfig):
1150         # Old signature: _execute_processing(req, progress=None)
1151         if req is None and db is not None and isinstance(db, ProcessRequest):
1152             req = db
1153             db = None
1154         elif req is None and isinstance(config, ProcessRequest):
1155             req = config
1156             config = None
1157         # Resolve config/db from module globals when not passed explicitly
1158         if config is None:
1159             config = global_config
1160         if db is None:
1161             db = db_instance
1162
1163     notify = progress or (lambda stage: None)
1164
1165     notify("hashing")
1166     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
1167     (identity ?
1168     # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
1169     original
1170     # req.video_path (the source URI) is kept for the DB record + report; only the file-
1171     reading
1172     # consumers (hash / validators / segmenter) use the resolved local path.
1173     local_video = storage.acquire_local_input(
1174         req.video_path, getattr(config, "storage", None)
1175     )
1176     video_id = compute_video_id(local_video)
1177     was_reingest = db.get_video(video_id) is not None
1178
1179     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
1180     report)
1181     notify("validating")
1182     logger.info(f"Running validations for {req.video_path}...")
1183     val_results, val_context = registry.run_all_with_context(local_video, config)
1184     failures = [r for r in val_results if not r.passed]
```

```

1179
1180 # typed AppConfig: attribute access for the default segmenter (with safe fallback)
1181 default_seg = getattr(
1182     getattr(config, "indexing", None), "default_segmenter", "motion"
1183 )
1184 seg_name = req.segmenter or default_seg
1185 segmenter_actual = None
1186 segmentation_ran = False
1187 # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
1188 # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
1189 # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
1190 compute_actual: Optional[str] = None
1191 response: Optional[Dict[str, Any]] = None
1192 try:
1193     if failures:
1194         # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
1195         # status; it no longer destroys the video's prior good segments.
1196         db.add_video(
1197             video_id,
1198             req.video_path,
1199             "failed",
1200             [r.model_dump() for r in val_results],
1201         )
1202         response = {
1203             "video_id": video_id,
1204             "status": "failed",
1205             "message": "Video failed validation checks.",
1206             "results": [r.model_dump() for r in val_results],
1207         }
1208         return response
1209
1210 # 2. Run segmenter & indexer
1211 logger.info("[API] Video passed checks. Segmenting and indexing...")
1212 db.add_video(
1213     video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
1214 )
1215
1216 segmenter_cls = segmenter_registry.get(seg_name)
1217 if not segmenter_cls:
1218     # Submit-time validation already 400s unknown names; this is the defensive
1219     # in-worker path (e.g. config default pointing at an unregistered
segmenter).
1220     available = list(segmenter_registry.list_available().keys())
1221     response = {
1222         "video_id": video_id,
1223         "status": "failed",
1224         "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
1225         "results": [r.model_dump() for r in val_results],
1226         "play_segments": [],
1227     }
1228     return response
1229
1230 logger.info(f"[API] Using segmenter: {seg_name}")
1231 notify(f"segmenting ({seg_name})")
1232 segmenter_actual = seg_name
1233 # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
1234 # if the run yields segments; otherwise drop the new partial rows and keep the
old.
1235 play_wm, cand_wm = db.segment_watermarks(video_id)
1236 # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,

```

```

run the heavy
1237         # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
1238         # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
1239         # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
1240         # they're dropped on the cloud path in this slice ? a documented follow-up.)
1241         remote = _maybe_dispatch_remote(
1242             config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
1243         )
1244         if remote is not None:
1245             response, segmentation_ran = (
1246                 remote # `ran` = did the cloud job actually execute (report)
1247             )
1248             return response # the cloud branch already stamped response["compute"]
(path=cloud_run)
1249         # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
1250         compute_actual = "in_process"
1251         segmenter = segmenter_cls(db, config)
1252         try:
1253             result = segmenter.process_video(
1254                 video_id, local_video, req.player_pool, req.max_frames
1255             )
1256         except Exception:
1257             # A raising segmenter must not leave half-written rows: restore the pre-run
1258             # state (same destroy-after-confirm contract as the Failure branch), then
1259             let
1260                 # the job worker record the failure.
1261                 db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
1262                 raise
1263             segmentation_ran = True
1264         if isinstance(result, Failure):
1265             logger.error(f"[API] Segmenter failed: {result.message}")
1266             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
1267             response = {
1268                 "video_id": video_id,
1269                 "status": "failed",
1270                 "message": f"Segmenter '{seg_name}' failed: {result.message}",
1271                 "results": [r.model_dump() for r in val_results],
1272                 "play_segments": [],
1273             }
1274             return response
1275
1276         segments = result.value if hasattr(result, "value") else result
1277         notify("finalizing")
1278         _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
1279
1280         response = {
1281             "video_id": video_id,
1282             "status": "success",
1283             "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
1284             "results": [r.model_dump() for r in val_results],
1285             "play_segments": segments,
1286         }
1287         return response
1288     finally:
1289         # Compute-path PROVENANCE stamp (the validation method): the cloud branch
already set

```

```

56:     enforce_startup_checks,
95:     ``request.app.state`` ? eliminating the Optional module-level globals that forced
321:     # SPA compute-picker (item 3): "local" forces in-process even on a cloud-configured
server;
322:     # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
712:     enforced, D11), or ``None`` for a cloud bucket (which is cost/quota-warned, never
blocked).
934:     are present so a forced per-request override can actually dispatch + collect a result.
995:     # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch (guarded
by
1172:     was_reingest = db.get_video(video_id) is not None
1327:         was_reingest=was_reingest,
1793: def _enforce_startup_or_exit(config) -> None:
1802:     enforce_startup_checks(config, logger=logger, include_exposure=True)
1983:     _enforce_startup_or_exit(
2059:         was_reingest = db_instance.get_video(video_id) is not None
2132:         was_reingest=was_reingest,

```

5. user (2026-07-02T02:58:29.588Z)

```

e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
52d97b5 P4: POST /api/assets ? store a video into the user's chosen medium (#409)
7ea6ea9 P3: /api/capabilities reports usable storage backends (#408)
f28af73 P2: engine free-space guard (HTTP 507 + WSL frame-stage BLOCK) (#407)
759a67f feat(api): frame-preserving annotation proxy for /api/source (D4) (#406)
5a5c4fa feat(api): P8a ? GET /api/source + POST /api/annotations (annotate-in-UI backend) (#405)
959c630 fix(#397): write per-video report to the resolved local path, not the raw URI (#400)
b6214e5 chore: ruff format the whole repo (259 files) (#387)
cd5b3d2 feat(01/P11): Gemini remote-file GC ? startup orphan sweep (#380)
9cd7598 fix(tests): resolve remaining test signature mismatches from app-factory refactor (#371)
de09ec8 refactor(main): app-factory + typed-config migration (D2/D3/P2/P3/P5/P7) (#370)
b25fdb8 docs: explain lazy cv2 import in --debug-clip path (Q8) (#361)
f523d3a refactor: enable isort (I) in ruff, auto-fix 143 files (Q6) (#360)
d92cc10 feat(serving): make the cloud compute-picker path work + provable (provenance +
skip_ai_handoff forward) (#341)

```

6. user (2026-07-02T02:58:39.480Z)

```

1288         finally:
1289             # Compute-path PROVENANCE stamp (the validation method): the cloud branch
already set
1290             # response["compute"] (path=cloud_run + the Cloud Run execution); for every
other path stamp
1291             # the actual path here so the job result PROVES which backend ran (in_process /
not_run) and
1292             # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
then visible.
1293             if isinstance(response, dict):
1294                 response.setdefault(
1295                     "compute",
1296                     {
1297                         "path": compute_actual or "not_run",
1298                         "requested": getattr(req, "compute", None),
1299                     },
1300                 )
1301             prov = response.get("compute", {}) if isinstance(response, dict) else {}
1302             logger.info(
1303                 "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
1304                 video_id,
1305                 getattr(req, "compute", None),
1306                 prov.get("path"),
1307                 prov.get("dispatched"),
1308                 (response or {}).get("status") if isinstance(response, dict) else None,

```

```

1309         )
1310         # Always emit the per-video observability report (next to the video, or to
1311         # report.output_dir). Never raises. Surface the paths in the response. (The
compute-path
1312         # proof lives on the job result + the [COMPUTE] log, not the report ?
obsreport's record_run
1313         # doesn't surface arbitrary run signals.)
1314         paths = emit_indexer_report(
1315             db=db,
1316             video_id=video_id,
1317             # Resolved LOCAL path, not the raw URI: obsreport writes the report next to
the
1318             # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://,
local://)
1319             # collapses to a literal scheme dir at the repo root (see #397). The source
URI is
1320             # still preserved in DB provenance via db.add_video.
1321             video_path=local_video,
1322             config=config,
1323             val_results=val_results,
1324             val_context=val_context,
1325             segmenter_requested=req.segmenter,
1326             segmenter_actual=segmenter_actual,
1327             was_reingest=was_reingest,
1328             segmentation_ran=segmentation_ran,
1329         )
1330         if paths and isinstance(response, dict):
1331             response["reports"] = paths
1332
1333
1334     def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
1335         """Best-effort size (bytes) of a remote storage source, or ``None`` if unknowable.
1336
1337         Used only to pre-reserve scratch for a fetch-to-local (P2). Never raises: a stat
that needs a
1338         missing cloud SDK, the network, or a credential returns ``None`` so the free-space
guard degrades
1339         to a no-op rather than blocking a job it cannot measure."""
1340         try:
1341             sc = (
1342                 storage_config.model_dump()
1343                 if hasattr(storage_config, "model_dump")
1344                 else (storage_config or {})
1345             )
1346             backend = storage.get_storage(input_ref.backend, config=sc)
1347             return int(backend.stat(input_ref).size)

```

7. user (2026-07-02T02:58:40.729Z)

```

badminton-highlight-indexer\backend\main.py:313:class ProcessRequest(BaseModel):
badminton-highlight-indexer\backend\main.py-314-    video_path: str
badminton-highlight-indexer\backend\main.py-315-    player_pool: Optional[List[str]] = None
badminton-highlight-indexer\backend\main.py-316-    max_frames: Optional[int] = None
badminton-highlight-indexer\backend\main.py-317-    segmenter: Optional[str] = None
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\main.py-318-    # The
UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Recorded on the job for PMF
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\main.py-319-    #
analytics (count_jobs_by_locale). Optional ? NULL when unrecorded (CLI / older clients).
badminton-highlight-indexer\backend\main.py-320-    locale: Optional[str] = None
badminton-highlight-indexer\backend\main.py-321-    # SPA compute-picker (item 3): "local"
forces in-process even on a cloud-configured server;
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\main.py-322-    #
"cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ? server
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\main.py-323-    #
default (deployment.compute_target). Only honoured for these two values; see

```

```

_maybe_dispatch_remote.
badminton-highlight-indexer\backend\main.py-324-     compute: Optional[str] = None
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py-325-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py-326-
badminton-highlight-indexer\backend\main.py-327-class AnnotationsRequest(BaseModel):
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py-328-     #
Finished rally labels for a video, as the annotator's byte-compatible CSV (P8, from the Studio
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py-329-     #
AnnotatePanel). `video_id` is the file MD5 (the pipeline's join key); `csv` is written verbatim.
badminton-highlight-indexer\backend\main.py-330-     video_id: str
badminton-highlight-indexer\backend\main.py-331-     csv: str
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py-332-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py-333-
badminton-highlight-indexer\backend\main.py-334-class AssetRequest(BaseModel):
badminton-highlight-indexer\backend\main.py-335-     # "Store this video in my medium"
(STUDIO_MEDIA_LIFECYCLE.md P4, D14). Exactly ONE source:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py-336-     #
uploaded_path ? a local file from the chunked upload (the durable copy's source; kept as the
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py-337-     #
local working copy for the run), OR
badminton-highlight-indexer\backend\main.py-338-     # source_uri ? a video ALREADY at rest
in a medium (a pasted s3:///gcs:///gdrive:// URI) ?

```

8. assistant (2026-07-02T02:58:48.815Z)

Code confirmed. Now let me check the GitHub issues and the segmenter default config.

9. user (2026-07-02T02:58:51.905Z)

```

{"body": "***Severity: cost + correctness.** A page refresh or the SPA's \"Try again\" re-uploads
the same file and POSTs `/api/process` again. Since `video_id` is the **MD5 of the file**, the
re-submit carries the **same `video_id`** ? but the engine **re-runs the full (paid) segmenter
anyway** instead of returning the existing result. On the Gemini path that's a repeated, real
API charge (? $0.10 per 8-min match) for zero new information, plus another multi-minute compute
run.\n\n### Confirmed current behavior (`backend/main.py`) \n`_execute_processing` computes
`was_reingest = db_instance.get_video(video_id) is not None` ? i.e. it **detects** the video is
already processed ? but uses that only for destroy-after-confirm safety (`_finalize_reingest`
prunes/replaces). There is **no `\"already processed ? reuse, skip the segmenter\"` path**: it
runs the segmenter and replaces the segments regardless. So a re-submit re-bills.\n\n### Live
trigger (alpha run, 2026-06-22)\nAn 8.3-min upload finished with 27 rallies, but the SPA's
10-min poll ceiling timed out **~seconds before completion** and showed `\"Couldn't process ? Try
again.\"` Clicking `\"Try again\"` would have re-uploaded the identical file and re-run Gemini end-
to-end. (We recovered by loading `?video=<id>` instead.) The SPA-side resume is tracked in
`avidullu/khelsutra#111`; **this issue is the engine-side guardrail** so we're protected even if
a re-submit slips through (defense in depth).\n\n### Proposed fix\n- Make `/api/process`
**idempotent by content hash**: if `video_id` already has valid `play_segments` and no explicit
`force`/`reprocess` flag, **return the existing result immediately** (the queryable `video_id`)
**without invoking the paid segmenter**.\n- Re-processing must be **opt-in** (`force=true` / an
explicit `\"Re-run\"` action) so it's never accidental.\n- Optional earlier short-circuit: if the
upload hashes to an existing `video_id`, skip the re-upload too.\n\n### Acceptance\n- [ ] Re-
submitting an already-processed video (same file/hash) returns the cached rallies with **zero**
segmenter/Gemini calls (assert no Gemini call in a test).\n- [ ] `force=true` still re-runs
(explicit), preserving the existing re-ingest destroy-after-confirm safety.\n- [ ] Default (no
force) is a no-op-cost reuse.\n- [ ] Pairs with `avidullu/khelsutra#111` (SPA resume so a
refresh doesn't re-trigger in the first place).\n\n**Cost context:** paid Gemini ? $0.10 per
8-min match; at alpha scale, accidental re-runs are real money + a nasty surprise on a metered
key. The M8 cost-ledger exists but doesn't meter Gemini usage yet, so a silent re-bill wouldn't
even show up.\""}, {"number": 340, "state": "OPEN", "title": "[cost] /api/process re-runs the paid
segmenter for an already-processed video ? refresh/\"Try again\" re-bills Gemini"}
----
{"number": 330, "state": "OPEN", "title": "[optimization] MD5 segmentation reuse ? skip Gemini re-
segmentation when the same video (video_id) is re-uploaded"}

```

10. user (2026-07-02T02:58:52.053Z)

```

badminton-highlight-indexer\config.json:26:         "default_segmenter": "gemini",
badminton-highlight-indexer\eval_baselines\reelcount_manifest.json:6:
"indexing.default_segmenter": "wasb_hybrid",
badminton-highlight-indexer\scripts\run_process_and_stitch.py:113:         seg_name =
cfg.indexing.default_segmenter
badminton-highlight-indexer\scripts\nightly_reelcount.py:563:         getattr(config, "indexing",
None), "default_segmenter", "motion"
badminton-highlight-indexer\backend\config\loader.py:256:#     * default_segmenter "motion" ?
pure-OpenCV, no torch / GPU / weights / API key (LIGHT tier);
badminton-highlight-indexer\backend\config\loader.py:264:     "indexing": {"default_segmenter":
"motion", "use_gpu": False},
badminton-highlight-indexer\tests\test_app_launcher.py:208:         assert
data["indexing"]["default_segmenter"] == "motion"
badminton-highlight-indexer\backend\config\models.py:150:     default_segmenter: str =
"yolo_hybrid"
badminton-highlight-indexer\backend\config\models.py:244:
@field_validator("default_segmenter")
badminton-highlight-indexer\tests\test_api_endpoints.py:123:     assert "default_segmenter" in
body
badminton-highlight-indexer\tests\test_api_endpoints.py:549:     assert "default_segmenter" in
body # readiness fields present for the SPA banner
badminton-highlight-indexer\backend\worker\__main__.py:256:         seg_name = args.segmenter or
config.indexing.default_segmenter
badminton-highlight-indexer\backend\worker\__main__.py:648:         "--segmenter", default=None,
help="default: indexing.default_segmenter"
badminton-highlight-indexer\backend\reporting\spec.yaml:133:         The segmenter that ran differs
from config.json's default_segmenter, and no
badminton-highlight-indexer\backend\reporting\harvest.py:244:     config_default =
(config.get("indexing") or {}).get("default_segmenter")
badminton-highlight-indexer\tests\test_config_deployment.py:177:     assert
cfg.indexing.default_segmenter == "yolo_hybrid" # untouched default
badminton-highlight-indexer\tests\test_config.py:24:     assert cfg.indexing.default_segmenter ==
"yolo_hybrid"
badminton-highlight-indexer\tests\test_config.py:90:def
test_served_default_segmenter_is_gemini():
badminton-highlight-indexer\tests\test_config.py:95:     assert cfg.indexing.default_segmenter ==
"gemini"
badminton-highlight-indexer\tests\test_config.py:106:     cfg = AppConfig(**{"indexing":
{"default_segmenter": "motion"}})

```

[Showing results with pagination = limit: 20]

11. user (2026-07-02T02:59:15.623Z)

```

badminton-highlight-indexer\backend\infrastructure\database.py:669:     def get_video(self,
video_id: str) -> Optional[Dict[str, Any]]:
badminton-highlight-indexer\backend\infrastructure\database.py:670-         with self._connect()
as conn:
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer\backend\infrastructure\database.py:671-         row = (
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer\backend\infrastructure\database.py:672-         conn.cursor()
--
badminton-highlight-indexer\backend\infrastructure\database.py:1018:     def get_video_cost(self,
video_id: str) -> Dict[str, Any]:
badminton-highlight-indexer\backend\infrastructure\database.py:1019-         """Aggregate cost
across a video's jobs: total `cost_usd` + the per-job rows. A video is
badminton-highlight-indexer\backend\infrastructure\database.py:1020-         1:1 with an infer
job, but a re-ingest can produce several ? sum them."""
badminton-highlight-indexer\backend\infrastructure\database.py:1021-         with self._connect()
as conn:
--
badminton-highlight-indexer\backend\main.py:1447:def get_video_cost(video_id: str, request:
Request):
badminton-highlight-indexer\backend\main.py:1448-         """Aggregate per-video cost across its jobs

```



```
(M8): ``total_cost_usd`` + ``total_cost_inr`` + the
badminton-highlight-indexer\backend\main.py-1449-    per-job rows. A video is 1:1 with an infer
job, but a re-ingest can produce several."""
C:\Users\avidu\Projects\khesutra-guru\badminton-highlight-indexer\backend\main.py-1450-    agg
= request.app.state.db.get_video_cost(video_id)
--
badminton-highlight-indexer\backend\utils\video.py:13:def get_video_duration(video_path: str) ->
float:
C:\Users\avidu\Projects\khesutra-guru\badminton-highlight-indexer\backend\utils\video.py-14-
"""Returns the exact duration of a video file in seconds using ffprobe.
C:\Users\avidu\Projects\khesutra-guru\badminton-highlight-indexer\backend\utils\video.py-15-
Returns 0.0 on failure."""
badminton-highlight-indexer\backend\utils\video.py-16-    try:
```

12. assistant (2026-07-02T02:59:27.133Z)

All checks complete. The claim reproduces exactly at HEAD.

13. user (2026-07-02T02:59:38.787Z)

Structured output provided successfully